2026-27-IT-G2

[SKIT]
ESTD. 2000

FaceForge: Multi-Modal Facial Emotion Recognition and Analytical Telemetry System

Software Requirements Specification

Version 2.0

Submitted in Partial Fulfillment for the Award of Degree of Bachelor of Technology in Information Technology from Rajasthan Technical University, Kota

MENTOR:
Dr. / Prof. Faculty Mentor
Dept. of Information Technology

SUBMITTED BY:
Member 4 (Lead Integrator & UI)
Roll No: 23ESKITXXX
In collaboration with Members 1, 2 & 3

COORDINATOR:
Project Coordinator
Dept. of Information Technology

PROJECT ID: 2026-27-IT-G2
TRACK: Computer Vision & Web Systems

DEPARTMENT OF INFORMATION TECHNOLOGY
SWAMI KESHWANAND INSTITUTE OF TECHNOLOGY, MANAGEMENT & GRAMOTHAN
(An Autonomous Institute Affiliated to Rajasthan Technical University, Kota)
Ramnagaria (Jagatpura), Jaipur – 302017

SESSION 2026–27

Revision History

Date	Version	Description	Author
20/07/2026	0.1	Initial requirement analysis, 4-member pipeline formulation, and system scope.	Member 4
01/10/2026	1.0	Draft SRS specifications with input/output contracts for Members 1, 2, and 3.	Member 4
31/10/2026	2.0	Revised SRS with complete Module 4 implementation (FastAPI backend, React Light UI, persistent results.csv storage, analytics suite, and test matrix).	Member 4

Table of Contents

1 Introduction **1**
1.1 Purpose 1
1.2 Motivation 1
1.3 Tools and Technologies 1
1.4 References 2
2 Market Survey **3**
2.1 Objectives 3
2.2 Unexplored Areas 3
2.3 Comparative Study of Similar Solutions 4
3 Requirements **5**
3.1 Functional Requirements 5
3.2 Non-Functional Requirements 5
3.3 Hardware Requirements 6
3.4 Software Requirements 6
3.5 Agile Model 7
3.6 Business Process Model 7
3.7 Supplementary Requirements 8
4 System Architecture **9**
4.1 Client-Server & 4-Layer Architecture 9
4.2 Communications Interfaces 9
5 Design and Implementation **10**
5.1 Product Features & 4-Member Division 10
5.2 Data Flow Diagrams (Level 0, 1 & 2) 11
5.3 E-R & Telemetry Storage Diagram 14
5.4 Class Diagram Design 14
5.5 Use Case Diagrams 15
5.6 Sequence Diagram 18
5.7 Component Diagram 19
6 Conclusion and Future Scope **20**
7 Mapping with UN Sustainable Development Goals **21**

1 Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to describe the technical architecture, system requirements, and implementation specifications for the multi-modal facial emotion recognition suite named **FaceForge**. The system is an engineering application designed to detect human faces, isolate facial regions of interest (ROI), and classify affective expressions from three operating inputs: static photographs, video files, and real-time live browser webcam streams.

The application is structured into four distinct modules to facilitate collaborative development across four team members. This specification is maintained by **Member 4** (Lead Integrator & UI Engineer) and provides complete operational specifications for the integration platform, while providing formal input/output interface contracts for Member 1 (Preprocessing), Member 2 (Computer Vision), and Member 3 (Emotion Model).

1.2 Motivation

Non-verbal affective cues, particularly facial expressions, represent over 55% of human interpersonal communication. While deep learning has made substantial strides in computer vision, deploying emotion recognition in practical applications remains challenging due to high computational latency on low-end hardware, lack of clean cross-platform web interfaces, and absent historical telemetry logging.

FaceForge addresses these barriers by combining OpenCV frame-sampling optimization, a pretrained facial expression model, an append-only telemetry storage ledger, and an accessible modern web interface. Rather than requiring users to install complex local Python environments, FaceForge provides an immediate, browser-accessible platform for affective analysis.

1.3 Tools and Technologies

The following tools and frameworks constitute the FaceForge tech stack:

- **Frontend (UI / Presentation Layer):** React 19, Vite, Vanilla CSS design tokens (Light theme), Lucide Icons.
- **Backend & API Gateway:** Python 3.11, FastAPI, Uvicorn ASGI server, Starlette StaticFiles.
- **Computer Vision & Image Processing:** OpenCV (cv2), NumPy array manipulation, Pillow (PIL).
- **Machine Learning & Emotion Engine:** Pretrained FER Deep Learning Model (CNN), Softmax activation.
- **Storage & Telemetry:** Append-only CSV storage engine (`storage.py` → `results/results.csv`), PyYAML.
- **Analytics & Visualization:** pandas, matplotlib, ReportLab PDF compilation.

1.4 References

1. Goodfellow, I., et al., *Challenges in Representation Learning: Facial Expression Recognition*, ICML, 2013.
2. OpenCV Open Source Computer Vision Library: https://opencv.org/
3. FastAPI Framework Documentation: https://fastapi.tiangolo.com/
4. React Documentation: https://react.dev/
5. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.

2 Market Survey

2.1 Objectives

The primary objectives of the market survey are: (1) To evaluate existing facial analysis toolkits; (2) To understand how pretrained deep learning networks perform on varied video sampling rates; (3) To identify pain points in multi-modal desktop applications; and (4) To contrast FaceForge against commercial and open-source benchmarks.

2.2 Unexplored Areas

While several facial emotion models exist on GitHub and HuggingFace, most existing tools are isolated Python scripts or Jupyter notebooks lacking practical deployment features:

• **Unified Multi-Modal Input:** Few systems seamlessly support Image, offline Video file sampling, and live browser camera streaming in one UI.

• **Historical Telemetry Logging:** Predictions are rarely preserved in queryable, structured CSV formats.

• **Lightweight Browser UI:** Existing tools often mandate heavier desktop GUI libraries (Tkinter/PyQt) rather than modern responsive web interfaces.

2.3 Comparative Study of Similar Solutions

Feature / Capability	Standard FER Scripts	DeepFace / Commercial	FaceForge (Our System)
Primary Interface	CLI / OpenCV window	Proprietary Cloud API	Modern React Light Web UI (HMR)
Multi-Modal Support	Static image only	Image / Cloud Video	Image, Video (Step Sampling), Webcam
Local Execution & Privacy	Local terminal only	Cloud upload required	100% On-Premise / Local Host
Persistent Telemetry	Console print only	Cloud dashboard database	Append-only results.csv ledger
Analytics Visualizer	None	Aggregated analytics	Distribution charts & PDF reports
Collaborative Modularity	Monolithic single script	Closed-source black box	4-Module Decoupled Architecture

3 Requirements

3.1 Functional Requirements (FRs)

• **FR-1 (Image Emotion Detection):** The system shall accept JPG and PNG image uploads, locate faces, render bounding boxes, and output the dominant emotion and confidence.

• **FR-2 (Video Frame Sampling):** The system shall process MP4 and AVI files by sampling every N-th frame (configurable 1-30) to reduce computation time while logging frame indices.

• **FR-3 (Real-Time Webcam Analysis):** The system shall capture live camera frames via the browser, evaluate facial emotions, and display a live floating HUD with active emotion and probability.

• **FR-4 (Persistent Telemetry Logging):** All inferences across all three modes shall be logged into `results/results.csv` with timestamp, source, frame number, face ID, emotion, and confidence.

• **FR-5 (Analytics & Statistics Engine):** The system shall calculate class frequencies across the 7 emotions, average confidence, and temporal affective timelines.

• **FR-6 (Config & Adapter Isolation):** The system shall provide feature toggles in `config.yaml` allowing mock adapters for Members 1, 2, and 3 to be enabled/disabled independently without crashing.

3.2 Non-Functional Requirements (NFRs)

• **Performance & Latency:** Single static image analysis shall complete in < 800 ms. Video processing with sampling step 10 shall maintain throughput > 20 FPS equivalent.

• **Usability & Design:** The user interface shall employ a clean light color palette with WCAG AA compliance and accessible high-contrast text.

• **Fault Tolerance & Robustness:** Detection requests with no faces or corrupt files shall return readable HTTP error messages without crashing the server process.

• **Data Integrity:** Storage operations shall append atomically to prevent file corruption during simultaneous requests.

3.3 Hardware Requirements

• **Development / Host System:** Intel Core i3 / AMD Ryzen 3 or higher, minimum 4 GB RAM, 2 GB free disk space.

• **Video Capture Device:** Integrated HD webcam or external USB camera supporting 720p / 30 FPS.

3.4 Software Requirements

- **Operating System:** Windows 10/11, Ubuntu 20.04+, or macOS.
- **Runtime Environments:** Python 3.10+ and Node.js v18+ with npm.
- **Core Libraries:** FastAPI, Uvicorn, OpenCV-Python, Pandas, Matplotlib, ReportLab.

3.5 Agile Model

FaceForge follows an iterative Agile lifecycle partitioned into 2-week sprints: Sprint 1 (Skeleton & Mock Adapters), Sprint 2 (REST API & Storage Engine), Sprint 3 (React UI & Light Theme), Sprint 4 (Analytics & Evaluation Integration), and Sprint 5 (Final Testing & Verification).

3.6 Business Process Model

1. User accesses web portal → 2. Selects input mode (Image/Video/Webcam) → 3. System acquires visual data → 4. Preprocessing prepares matrix → 5. Face detector extracts bounding boxes → 6. Pretrained FER outputs emotion vector → 7. Results appended to CSV ledger → 8. Visual overlays and analytics updated.

4 System Architecture

4.1 Client-Server & 4-Layer Architecture

FaceForge employs a decoupled client-server architecture consisting of a React Single-Page Application (SPA) client, a FastAPI backend application server, a persistent file-based CSV telemetry store, and modular adapter pipelines. The architecture is organized across four collaborative layers:

- **Layer 1 (Input Layer):** Interfaces with user files (Image/Video) and client camera devices.
- **Layer 2 (Computer Vision Layer):** OpenCV matrix decoding, resizing, grayscale, face ROI cropping.
- **Layer 3 (AI / Emotion Layer):** Pretrained FER model generating probability distributions across 7 emotions.
- **Layer 4 (Integration & UI Layer):** FastAPI API Gateway, React UI, results storage, and analytics visualizer.

4.2 Communications Interfaces

- **Client-Server REST APIs:** HTTP/1.1 JSON-based exchange between React frontend and FastAPI on port 8000.
- **File Upload Protocol:** Multipart form-data encoding for image and video transfers.
- **Webcam Streaming Protocol:** Canvas frame capture with base64/binary POST streaming.

5 Design and Implementation

5.1 Product Features & 4-Member Collaborative Division

5.1.1 Module 1: Preprocessing & Dataset Specifications (Member 1)

[ALLOCATED RESPONSIBILITY: MEMBER 1 — DATASET & PREPROCESSING]
- **Input Contract:** Raw visual media & FER-2013 directory partitions.
- **Output Contract:** Cleaned, normalized, resized grayscale/RGB image matrices.
- *Status:* Interface contract established in adapter_preprocess.py. Member 1 is responsible for populating specific dataset partition statistics, histogram equalization parameters, and tensor normalization formulas.

5.1.2 Module 2: Computer Vision & Face Detection Specifications (Member 2)

[ALLOCATED RESPONSIBILITY: MEMBER 2 — COMPUTER VISION & DETECTION]
- **Input Contract:** Decoded image matrix or video stream frame from OpenCV.
- **Output Contract:** Face bounding box coordinates (x, y, w, h) and cropped face ROI image[y:y+h, x:x+w].
- *Status:* Interface contract established in adapter_face.py. Member 2 is responsible for finalizing the face detector model, multi-face array loops, video frame sampling mechanics, and camera FPS stabilization.

5.1.3 Module 3: Pretrained Emotion Model & Evaluation Specifications (Member 3)

[ALLOCATED RESPONSIBILITY: MEMBER 3 — PRETRAINED MODEL & EVALUATION]
• **Input Contract:** Extracted face ROI matrix.
• **Output Contract:** 7-class emotion probabilities and dominant label with confidence score.
• *Status:* Interface contract established in adapter_emotion.py. Member 3 is responsible for integrating the pretrained FER network, generating confusion_matrix.png, and compiling Precision/Recall/F1 in metrics_report.txt.

5.1.4 Module 4: System Integration, UI, Storage & Analytics (Member 4 — Fully Active)

[ALLOCATED RESPONSIBILITY: MEMBER 4 — FULLY IMPLEMENTED & ACTIVE]
• **Application Orchestrator (`app.py`):** Central FastAPI server exposing 10 REST endpoints with CORS and static SPA mounting.
• **Light-Themed Web UI (`frontend/`):** Responsive React interface covering Home, Image, Video, Webcam, Results Ledger, and Analytics Dashboard.
• **Telemetry Ledger Engine (`storage.py`):** Append-only storage writing detections to results/results.csv with filter/export/clear operations.
• **Statistical Analytics Visualizer (`analytics.py`):** Computes class distribution, temporal valence tracking, and mean confidence vectors.

5.2 Data Flow Diagrams (DFDs)

DFD Level 0 (Context Diagram):

```
[User / Client] ↔ (Input: Image/Video/Webcam) ↔ [0.0 FaceForge Vision System] ↔ (Output: Overlays, CSV Records, Analytics Charts)
```

DFD Level 1 (Major Subsystems):

```
[User] → [1.0 Media Acquisition] → [2.0 Preprocessing (M1)] → [3.0 Face Detection (M2)] → [4.0 Emotion Inference (M3)] → [5.0 Storage & Analytics (M4)] → [results.csv]
```

DFD Level 2 (Module 4 Pipeline & Storage Flow):

```
[API Route: /api/detect/*] → [controller.py Dispatcher] → [Mock/Live Adapters] → [storage.log_detection()] → [results.csv] → [analytics.compute_analytics()] → [React UI View]
```

5.3 Entity-Relationship & Telemetry Storage Schema (`storage.py`)

The primary persistent entity is the inference record appended to `results.csv`:

Attribute	Data Type	Constraints	Description
timestamp	VARCHAR(19)	NOT NULL	Format: YYYY-MM-DD HH:MM:SS
source	VARCHAR(10)	CHECK(source IN ('image','video','webcam'))	Input medium category
file_name	VARCHAR(255)	NOT NULL	Source file name or camera index
frame_number	INTEGER	DEFAULT 0	Frame index in video sequence
face_id	INTEGER	DEFAULT 0	Index of face in multi-face frame
emotion	VARCHAR(20)	NOT NULL	Classified expression label
confidence	FLOAT	CHECK(confidence BETWEEN 0 AND 1)	Model probability scalar
status	VARCHAR(25)	NOT NULL	Affective group: Happy, Neutral, etc.
data_source	VARCHAR(10)	CHECK(data_source IN ('live','demo'))	Data provenance flag

5.4 Class & Modular Component Architecture

The Python backend is structured into decoupled modules:
• controller.py: Coordinates pipeline execution, loads configuration, annotates images with bounding boxes.
• storage.py: Manages CSV initialization, atomic appending, filtering, and resetting.
• analytics.py: Performs pandas aggregations, computes class distributions, and creates matplotlib figures.
• adapter_preprocess.py, adapter_face.py, adapter_emotion.py: Teammate adapter boundary wrappers.

5.5 Use Case & Sequence Workflow

Primary Use Cases: (1) UC-1: Upload & Analyze Image; (2) UC-2: Process Video File with Frame Sampling; (3) UC-3: Stream Live Webcam Inference; (4) UC-4: Filter & Export Results CSV; (5) UC-5: Generate Analytics PDF Report; (6) UC-6: Configure Adapters & Confidence Thresholds.

Sequence of Operations: Client uploads file → FastAPI receives multipart request → Controller dispatches to Preprocess Adapter (M1) → Face Detector Adapter (M2) locates bounding box → Emotion Adapter (M3) evaluates face → Storage Engine records row to CSV → JSON returned to React UI → Bounding box rendered on canvas.

6 Conclusion and Future Scope

Conclusion: FaceForge provides an enterprise-ready affective vision architecture. By clearly establishing decoupled module boundaries, Members 1, 2, and 3 are enabled to test and plug in their datasets, detectors, and evaluation models independently, while Member 4 delivers a fully functional web application, persistent storage ledger, and analytical reporting suite.

Future Scope: (1) Custom model fine-tuning on domain-specific emotional expressions; (2) GPU CUDA acceleration for multi-stream 60 FPS feeds; (3) Docker containerization for one-click cross-platform deployment; (4) Mobile camera PWA support.

7 Mapping with UN Sustainable Development Goals

The following table maps FaceForge to the United Nations Sustainable Development Goals (SDGs):

SDG No.	SDG Title	Relevance to FaceForge Project
SDG 3	Good Health and Well-being	Facial expression monitoring can provide non-invasive clinical mental health screening, stress detection, and patient affective state tracking.
SDG 4	Quality Education	Enables automated learner engagement analysis in virtual and physical classrooms, alerting educators to student confusion or disinterest.
SDG 9	Industry, Innovation, and Infrastructure	Applies digital innovation by uniting computer vision, pretrained neural networks, and modular web architecture on accessible desktop hardware.
SDG 10	Reduced Inequalities	The browser-based light UI provides open access to affective computing tools without requiring expensive cloud subscriptions or proprietary hardware.